TECHNICAL INTERNSHIP PROJECT REPORT

Cloud-Based Personal Expense Tracker using AWS & DevOps

Production-Grade Architecture, Microservices Containerization, CI/CD Pipeline Automation, and Cloud Infrastructure

TECHNOLOGY DOMAIN

Cloud Computing, DevOps Engineering & Full-Stack Systems

Document Type:	**Final Internship Technical Project Report**
Application Stack:	**React 19, Node.js, Express, MySQL, Docker, Kubernetes**
Cloud Infrastructure:	**Amazon Web Services (S3, CloudFront, EC2, RDS, CloudWatch, SNS, IAM)**
DevOps & CI/CD:	**GitHub Actions, Docker Compose, Kubernetes (K8s), Helm**
Submission Target:	**AWS Cloud & DevOps Engineering Internship Evaluation**
Document Status:	**Final Production Release (v1.0.0)**

Table of Contents

1. Executive Abstract

The **Cloud-Based Personal Expense Tracker** is an enterprise-grade, secure, multi-tier web application designed to empower individual users with real-time financial tracking, categorical budgeting, dynamic expense analytics, and automated report generation. In an increasingly digital economy, personal financial management remains severely hindered by fragmented manual tracking, lack of actionable spending visualization, and data vulnerability risks associated with unencrypted offline tools. This project addresses these challenges by delivering a cloud-native, highly available, and auto-scalable financial platform built upon modern web frameworks and AWS infrastructure.

The application architecture adopts a strict separation of concerns utilizing a decoupled full-stack model: a high-performance frontend built with **React 19, TypeScript, Vite, and Tailwind CSS**; an asynchronous, RESTful microservices API driven by **Node.js and Express.js**; and an enterprise-grade relational persistence tier powered by **Amazon RDS MySQL**. System security is engineered directly into the core application layer through JSON Web Tokens (JWT) for stateless authentication, bcrypt password hashing, HTTP header hardening via Helmet, Cross-Origin Resource Sharing (CORS) enforcement, automated rate-limiting, and SQL injection prevention via parameterized queries.

To achieve seamless elasticity and operational resilience, the platform leverages advanced DevOps practices. Containerization is implemented via **Docker and Docker Compose**, while production deployment is orchestrated using **Kubernetes (EKS/K8s)** with declared Ingress controllers, ConfigMaps, and Secrets. Infrastructure delivery is entirely automated through a continuous integration and continuous deployment (CI/CD) pipeline built on **GitHub Actions**. Cloud deployment spans **Amazon S3** for static asset delivery and receipt blob storage, **Amazon CloudFront** for global content delivery Network (CDN) edge caching, an **Application Load Balancer (ALB)** for routing, **Amazon EC2** instances running backend containers, **Amazon CloudWatch** for telemetry and logging, and **Amazon SNS** for threshold-based user alerts. This project demonstrates end-to-end cloud engineering proficiency, combining software engineering excellence with cloud architecture standards suitable for production-scale deployment.

2. Introduction & Industry Context

2.1 Expense Management Challenges in the Digital Age

Personal financial governance is a cornerstone of individual financial stability. However, modern consumers face unprecedented complexity due to the fragmentation of payment channels, including credit cards, digital wallets, bank transfers, and cash transactions. Without centralized, real-time logging and classification, individuals suffer from financial leakage—unnoticed recurring micro-transactions, impulse spending, and budget overruns. Traditional tracking mechanisms (e.g., paper notebooks or physical receipts) are inherently error-prone, lack real-time computation, and offer zero data redundancy.

2.2 Importance of Digital and Automated Expense Tracking

Digital expense tracking systems convert raw transactional data into structured, actionable intelligence. By categorizing individual cash flows, computing running totals, and rendering interactive visual metrics, digital tools provide users with immediate situational awareness of their net liquidity and spending velocity. Automated reporting

eliminates calculation friction, enabling users to establish disciplined spending thresholds, plan long-term savings, and generate structured financial statements for personal auditing and tax preparation.

2.3 The Pivotal Role of Cloud Computing in Modern Applications

Transitioning personal finance software from local desktop or mobile-only storage to cloud infrastructure provides distinct architectural benefits:

- **Universal Accessibility:** Users can securely interact with their financial data across heterogeneous endpoints (desktops, tablets, mobile browsers) with centralized state synchronization.
- **High Availability & Reliability:** Multi-AZ (Availability Zone) cloud deployments guarantee uptime, ensuring that application endpoints remain accessible even during localized hardware failures.
- **Durability and Data Protection:** Cloud-native databases and object storage provide automatic managed backups, automated point-in-time recovery (PITR), and replication, eliminating local hardware failure data loss.
- **Elastic Resource Scaling:** Cloud-hosted infrastructure dynamically scales computing resources to handle unpredictable usage spikes without manual hardware provisioning.

2.4 Need for Secure Financial Applications

Financial records represent highly sensitive Personal Identifiable Information (PII). A breach of user transaction logs compromises personal privacy and exposes behavioral patterns. Consequently, modern financial applications must enforce rigorous zero-trust security postures. This includes enforcing end-to-end data encryption in transit (TLS 1.3) and at rest (AES-256), robust cryptographic password hashing, stateless token management, and continuous infrastructure auditing to prevent unauthorized data exfiltration or system tampering.

3. Problem Statement & Requirements Engineering

Despite the abundance of commercial financial tools, traditional and entry-level expense tracking solutions exhibit systemic flaws that severely limit their security, usability, and scale:

- **Manual Record-Keeping & Human Error:** Manual ledger logging is highly susceptible to missing entries, miscalculations, and inconsistent categorization, rendering historical data unreliable.
- **Catastrophic Data Loss Risks:** Spreadsheet files (e.g., CSV/XLSX stored on local drives) or physical notebooks lack offsite replication. Physical device damage, disk corruption, or accidental deletion results in permanent data destruction.
- **Absence of Real-Time Analytics & Visual Insights:** Raw tabular data fails to highlight spending trends. Without dynamic multi-axis data visualizations (pie charts, bar graphs, historical trends), users cannot easily identify problematic expense categories.
- **Restricted Platform Accessibility:** Local desktop software restricts data interaction to a single physical terminal, preventing real-time, on-the-go logging at point-of-sale locations.

- **Poor Scalability & Security Flaws:** Off-the-shelf basic web tools often lack rigorous security controls, utilizing unencrypted database connections, plain-text passwords, or monolithic single-server architectures that fail under high concurrent loads.

4. Project Objectives & Scope

The objective of this project is to architect, build, secure, and deploy a cloud-native Personal Expense Tracker application using AWS services and modern DevOps methodologies.

Core Engineering Objectives

- **Full-Stack Application Development:** Architect a responsive React 19 web interface backed by a Node.js Express REST API and Amazon RDS MySQL persistence layer.
- **Enterprise Security Architecture:** Implement stateless JWT authentication, bcrypt password encryption, input validation, rate limiting, and HTTP header security.
- **Cloud-Native Deployment:** Deploy application tiers on AWS using Amazon S3, CloudFront CDN, Application Load Balancers, EC2, and managed RDS databases.
- **DevOps & Orchestration Automation:** Containerize application components using Docker, define multi-service orchestration with Docker Compose and Kubernetes manifests, and automate delivery via GitHub Actions CI/CD pipelines.
- **Telemetry & Alerting Infrastructure:** Establish system monitoring via Amazon CloudWatch and configure threshold alerts utilizing Amazon SNS.

5. Analysis of Existing Systems vs. Proposed Architecture

A comparative analysis highlights the architectural transition from legacy tracking methods to the modern cloud-native system engineered in this project.

Dimension	Legacy / Traditional System	Proposed Cloud-Native Platform
Data Persistence	Local file storage (Excel, CSV) or physical paper ledgers.	Amazon RDS MySQL database with automated multi-AZ replication.
Accessibility	Restricted to a single physical machine or location.	Global multi-device web access optimized via Amazon CloudFront CDN.
Security Profile	Unencrypted local files; vulnerable to physical theft/loss.	JWT stateless tokens, bcrypt encryption, Helmet, CORS, and SSL/TLS.
Analytics & Metrics	Manual aggregation; static charts requiring manual updates.	Automated visual dashboard, dynamic charts, and real-time category reports.
Receipt Management	Physical paper receipts; easy to misplace or damage.	Cloud object storage (Amazon S3) with secure pre-signed asset URLs.

Dimension	Legacy / Traditional System	Proposed Cloud-Native Platform
Deployment & Scaling	Manual deployment; static single-instance bound to hardware.	Automated GitHub Actions CI/CD pipeline targeting Kubernetes (K8s) clusters.
System Telemetry	Zero runtime visibility; manual OS-level debugging required.	Real-time telemetry and alarm notifications via CloudWatch and SNS.

6. Comprehensive Features & Functional Modules

6.1 Authentication & User Access Module

Manages complete user lifecycle operations with strict security standards.

- **User Registration:** Enforces strong password rules, validates email uniqueness, hashes credentials via bcrypt, and provisions database user records.
- **Stateless User Login:** Authenticates credentials against stored hashes and issues a cryptographically signed JWT containing user claims and expiration timestamps.
- **JWT Token Refresh & Logout:** Secure client-side token disposal combined with middleware token verification on all protected routes.
- **Profile Management:** Enables users to retrieve and update account details, profile metadata, and security parameters.

6.2 Dashboard & Financial Analytics Module

Provides a real-time command center for personal finance management.

- **Aggregated Summary Metrics:** Displays dynamic KPIs including Lifetime Expense Total, Current Month Spend, YTD Spend, and Average Transaction Value.
- **Interactive Data Visualization:** Integrated Chart.js/Recharts modules rendering monthly spending trends (line charts) and category distributions (doughnut charts).
- **Recent Transaction Feed:** Real-time tabular list of the latest transactions with quick inline action triggers (edit/delete).

6.3 Expense Management Core Module

Facilitates complete CRUD lifecycle management for financial transactions.

- **Add Expense:** Accepts transaction amount, category binding, transaction timestamp, payment method (Credit Card, Debit Card, UPI, Cash, Bank Transfer), and optional S3 receipt uploads.
- **Update Expense:** Allows state modifications for existing transaction logs with automatic database sync and re-indexing.
- **Delete Expense:** Soft and hard deletion options with immediate dashboard metric recalculation.
- **Multi-Parametric Search & Filtering:** Real-time filtering by date ranges, category IDs, payment modes, and keyword descriptions.

6.4 Category & Master Data Module

Organizes transactional records into granular categories for analysis.

- **Pre-Defined System Categories:** Out-of-the-box system taxonomy including *Food & Dining, Travel & Transport, Shopping, Bills & Utilities, Medical & Healthcare, Entertainment, Education, Investments,* and *Miscellaneous*.
- **Custom User Categories:** Allows creation, modification, and deletion of user-defined custom spending categories.

6.5 Reporting & Export Module

Generates detailed analytical audit trails and downloadable summaries.

- **Monthly & Yearly Reports:** Computes period-over-period delta variations and spending speed metrics.
- **Category Variance Analysis:** Breaks down absolute spend against user-defined budget caps with visual warning thresholds.
- **Data Export Engines:** Enables client-side and server-side compilation of transaction history into CSV and structured PDF reports.

7. System Architecture & AWS Cloud Topology

The application architecture utilizes a cloud-native, multi-tier layout designed for high availability, security isolation, and independent scalability of application layers.

AWS Cloud Deployment Architecture (Topology Overview)

```
[Client Browser / Mobile Endpoint] | ▼ [Amazon CloudFront CDN (Edge Caching & SSL Termination)] |
┌──────────────────────────────────────┐ ▼ ▼ [Amazon S3 Bucket] (Static Web App) [Application Load
Balancer (ALB)] | ▼ [Public Subnet / EC2 Cluster] ┌──────────────────────────┐ | Node.js Express
Backend API | | (Docker Containers / K8s) | └──────────┬──────────┘ |
┌──────────────────────┼──────────────────────┐ ▼ ▼ ▼ [Amazon RDS MySQL (DB)] [Amazon S3
(Receipts)] [Amazon CloudWatch / SNS] (Private DB Subnet) (Private Bucket) (Logging & Alerts)
```

7.1 AWS Deployment Architecture Details

- **Edge Layer & Static Asset Distribution:** The React 19 single-page application (SPA) is built using Vite and deployed to an Amazon S3 bucket configured for static web hosting. Amazon CloudFront sits in front of S3, caching assets globally across Edge Locations to reduce latency and provide TLS/SSL termination.
- **Ingress & Load Balancing:** Dynamic API traffic routed to `api.expensetracker.internal` passes through an AWS Application Load Balancer (ALB), which handles SSL certificates, health-checking, and traffic distribution across backend compute nodes.
- **Compute Layer:** The backend Express.js microservices run inside Docker containers deployed on Amazon EC2 instances managed within an Auto Scaling Group (ASG) or an Amazon Elastic Kubernetes Service (EKS) pod cluster across multiple Availability Zones.
- **Database Layer:** Persistence is handled by Amazon RDS for MySQL located inside isolated private database subnets. Multi-AZ replication ensures real-time synchronous data mirroring for high availability and failover resilience.
- **Object Storage & Telemetry:** Receipts and user attachment files are written directly to an Amazon S3 Receipt Bucket using AWS SDK pre-signed URLs. System logs, container stdout, and API performance metrics stream into Amazon CloudWatch, triggering Amazon Simple Notification Service (SNS) alerts when memory or error thresholds are breached.

7.2 Mermaid System Architecture Diagram

Mermaid Diagram Code: System Architecture Architecture

```
graph TD User([User Browser / Mobile]) -->|1. HTTPS Request| CF[Amazon CloudFront CDN] CF -->|2.
Fetch Static Assets| S3Web[Amazon S3: Web Hosting] User -->|3. REST API Calls| ALB[Application Load
Balancer] ALB -->|4. Forward Request| EC2[Amazon EC2 / K8s Container] subgraph Compute Layer EC2 -->|
Node.js / Express API| App[Expense Application Engine] end subgraph Data & Storage Tier App -->|5.
SQL Queries| RDS[(Amazon RDS MySQL Master)] RDS -.->|Multi-AZ Sync| RDSReplica[(Amazon RDS Standby)]
App -->|6. Upload Receipts| S3Data[Amazon S3: Receipt Storage] end subgraph Operations & Security App
-->|7. Telemetry & Logs| CW[Amazon CloudWatch] CW -->|8. Alarm Trigger| SNS[Amazon SNS Notification
Service] SNS -->|9. Admin Alert| Admin([DevOps Admin Email/SMS]) end
```

8. Database Design & Schema Architecture

The database architecture uses a relational model designed in third normal form (3NF) to preserve data integrity, eliminate redundancy, and support performant queries.

8.1 Entity Definitions and Data Dictionary

Table 1: `users`

Stores primary user authentication records, credentials, and metadata.

Column Name	Data Type	Constraints	Description
`user_id`	INT / BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each user account.
`name`	VARCHAR(100)	NOT NULL	Full legal name of the user.
`email`	VARCHAR(150)	NOT NULL, UNIQUE, INDEX	Unique email address used for system login.
`password`	VARCHAR(255)	NOT NULL	Cryptographic password hash produced via bcrypt.
`created_at`	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	System record creation timestamp.

Table 2: `categories`

Contains system default and custom expense categories.

Column Name	Data Type	Constraints	Description
`category_id`	INT	PRIMARY KEY, AUTO_INCREMENT	Unique primary key for category records.
`category_name`	VARCHAR(50)	NOT NULL, UNIQUE	Human-readable category title (e.g., Food, Travel).

Table 3: `expenses`

Stores granular transactional entries linked to users and categories.

Column Name	Data Type	Constraints	Description
`expense_id`	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique primary key for transaction records.

Column Name	Data Type	Constraints	Description
`user_id`	INT	NOT NULL, FOREIGN KEY (users.user_id)	Foreign key mapping transaction to owning user.
`category_id`	INT	NOT NULL, FOREIGN KEY (categories.category_id)	Foreign key binding transaction to category.
`amount`	DECIMAL(10, 2)	NOT NULL, CHECK (amount > 0)	Financial transaction value in standard currency.
`description`	TEXT	NULLABLE	Detailed memo or notes describing the item.
`payment_method`	ENUM(...)	NOT NULL, DEFAULT 'Cash'	Method: 'Cash','Credit Card','Debit Card','UPI','Bank Transfer'
`receipt_url`	VARCHAR(512)	NULLABLE	Amazon S3 Object URL for receipt images.
`expense_date`	DATE	NOT NULL, INDEX	Date when the expenditure occurred.
`created_at`	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record insertion timestamp.

8.2 Relational Integrity & Schema Constraints

- **Foreign Key Integrity:** `expenses.user_id` references `users.user_id` with `ON DELETE CASCADE` to clean up dependencies upon account termination. `expenses.category_id` references `categories.category_id` with `ON DELETE RESTRICT` to prevent orphan categories.
- **Indexing Strategy:** Composite index added on `(user_id, expense_date)` to accelerate range queries for monthly reports and dashboard aggregations.

9. Entity-Relationship (ER) Diagram

The ER diagram below represents the logical entities, primary/foreign key mappings, and cardinality constraints defining the application database structure.

Mermaid Diagram Code: Entity-Relationship Diagram

```
erDiagram USERS ||--o{ EXPENSES : "creates and owns" CATEGORIES ||--o{ EXPENSES : "classifies" USERS { int user_id PK "Auto Increment" string name "User Full Name" string email UK "Unique Login Email" string password "Bcrypt Encrypted Hash" timestamp created_at "Account Creation Time" } CATEGORIES { int category_id PK "Auto Increment" string category_name UK "Unique Category Name" } EXPENSES { bigint expense_id PK "Auto Increment" int user_id FK "References USERS(user_id)" int category_id FK "References CATEGORIES(category_id)" decimal amount "Transaction Value (10,2)" string description "Transaction Memo" string payment_method "Enum: Cash, Card, UPI, etc." string receipt_url "S3 Asset Reference URL" date expense_date "Transaction Date" timestamp created_at "System Log Timestamp" }
```

10. Unified Modeling Language (UML) Diagrams

10.1 Use Case Diagram

Defines user interactions and functional capabilities within the boundary of the system.

Mermaid Diagram Code: Use Case Diagram

```
graph LR User((User)) subgraph Expense Tracker System Boundary UC1(Register Account) UC2(Authenticate / Login) UC3(Manage Profile) UC4(Add Expense) UC5(Update / Delete Expense) UC6(View Dashboard Metrics) UC7(Generate Category Reports) UC8(Upload Receipt Attachment) end User --> UC1 User --> UC2 User --> UC3 User --> UC4 User --> UC5 User --> UC6 User --> UC7 UC4 ..> UC8 : <<Includes>>
```

10.2 Data Flow Diagram (DFD) - Level 0 (Context Diagram)

Illustrates system boundaries, external entities, and top-level data flow vectors.

Mermaid Diagram Code: DFD Level 0

```
graph TD User([User Entity]) <-->|User Registration / Credentials / JWT Tokens| System[Cloud Expense Tracker Platform] System <-->|SQL Queries / Persisted Records| DB[(Amazon RDS MySQL Database)] System <-->|Receipt Binary File / Object Storage S3 URL| S3[(Amazon S3 Object Storage)] System -->|Logs / Telemetry Stream| CloudWatch[Amazon CloudWatch Monitoring]
```

10.3 Data Flow Diagram (DFD) - Level 1

Decomposes internal processes, data transformations, and specific datastore interactions.

Mermaid Diagram Code: DFD Level 1

```
graph TD User([User]) -->|1. Auth Credentials| P1[1.0 Authentication Process] P1 -->|Validate & Hash| DS1[(Users Store)] P1 -->|Return JWT Token| User User -->|2. Transaction Data| P2[2.0 Expense Processing] P2 -->|Verify JWT Middleware| AuthCheck{Authorized?} AuthCheck -->|Yes| P2_Store[Write Expense] P2_Store -->|Insert/Update SQL| DS2[(Expenses Store)] User -->|3. Receipt File| P3[3.0 Storage Manager] P3 -->|AWS SDK Binary Put| DS3[(S3 Receipt Bucket)] DS3 -->|Return Asset URL| P2_Store User -->|4. Request Analytics| P4[4.0 Aggregation Engine] DS2 -->|Read Raw Data| P4 P4 -->| Format JSON Metrics & Charts| User
```

10.4 Sequence Diagram: Authentication & Expense Creation

Details time-ordered message interactions across client, backend API, database, and AWS object storage.

Mermaid Diagram Code: Sequence Diagram

```
sequenceDiagram autonumber actor Client as User (React SPA) participant API as Backend API (Express.js) participant DB as Database (RDS MySQL) participant S3 as Storage (Amazon S3) Client->>API: POST /api/v1/auth/login (email, password) API->>DB: SELECT * FROM users WHERE email = ? DB-->>API: User Record + Bcrypt Password Hash API->>API: Verify Password via bcrypt.compare() API-->>Client: 200 OK + Signed JWT Token Note over Client, API: Submitting New Expense Record with Receipt Attachment Client->>API: POST /api/v1/expenses (Form-Data + Bearer JWT) API->>API: Validate Token & Payload Schema alt Receipt Image Attached API->>S3: PutObject Command (Receipt Buffer) S3-->>API: 200 OK (S3 Object Key/URL) end API->>DB: INSERT INTO expenses (user_id, amount, category_id, receipt_url, ...) DB-->>API: Insert Result (expense_id = 1048) API-->>Client: 201 Created (JSON Response Payload)
```

11. RESTful API Documentation & Interface Specification

The Express.js backend exposes a RESTful JSON API. All protected routes require a standard HTTP Authorization header carrying a valid JWT Bearer token: `Authorization: Bearer <token>`.

11.1 Authentication Endpoint Matrix

`POST /api/v1/auth/register`

Registers a new user within the system.

```
// Request Body Schema
{
  "name": "Jane Doe",
  "email": "jane.doe@example.com",
  "password": "SecurePassword123!"
}

// Success Response (201 Created)
{
  "status": "success",
  "message": "User registered successfully",
  "data": {
    "user_id": 42,
    "name": "Jane Doe",
    "email": "jane.doe@example.com"
  }
}
```

`POST /api/v1/auth/login`

Authenticates user credentials and issues a JWT token.

```
// Request Body Schema
{
  "email": "jane.doe@example.com",
  "password": "SecurePassword123!"
}

// Success Response (200 OK)
{
  "status": "success",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "user_id": 42,
    "name": "Jane Doe",
    "email": "jane.doe@example.com"
  }
}
```

11.2 Expense Core Endpoint Matrix

`GET /api/v1/expenses`

Retrieves paginated, filtered transaction records for the authenticated user.

Query Parameters: `page=1`, `limit=10`, `startDate=2026-01-01`, `endDate=2026-01-31`, `categoryId=2`

```
// Success Response (200 OK)
{
  "status": "success",
  "pagination": { "total_records": 1, "page": 1, "total_pages": 1 },
  "data": [
    {
      "expense_id": 1048,
      "amount": 145.50,
      "description": "Weekly Groceries",
      "payment_method": "Credit Card",
      "category_name": "Food & Dining",
      "receipt_url": "https://expense-tracker-receipts.s3.amazonaws.com/receipts/rec_1048.jpg",
      "expense_date": "2026-01-15"
    }
  ]
}
```

`POST /api/v1/expenses`

Creates a new expense transaction entry.

```
// Request Body (Multipart / JSON)
{
  "category_id": 2,
  "amount": 145.50,
  "description": "Weekly Groceries",
  "payment_method": "Credit Card",
  "expense_date": "2026-01-15"
}

// Success Response (201 Created)
{
  "status": "success",
  "message": "Expense created successfully",
  "expense_id": 1048
}
```

`PUT /api/v1/expenses/:id` & **`DELETE /api/v1/expenses/:id`**

- `PUT /api/v1/expenses/:id` : Modifies specified fields of an existing expense record. Returns `200 OK`.
- `DELETE /api/v1/expenses/:id` : Removes the specified record. Returns `200 OK` with confirmation message.

11.3 Category & Analytics Endpoints

- `GET /api/v1/categories` : Fetches system and user categories. Returns `200 OK`.

- `POST /api/v1/categories` : Provisions a custom category record. Returns `201 Created` .
- `GET /api/v1/dashboard` : Aggregates metrics for rendering dashboard visualizations. Returns lifetime total, current month total, and category breakdowns.

12. Security Implementation & Hardening Measures

Security controls are implemented across all application layers to prevent unauthorized access and data breaches:

12.1 Authentication & Password Cryptography

- **Stateless JWT Validation:** User auth states rely on cryptographic JWT signatures using HMAC-SHA256 algorithm keys. Tokens include standard claims (`sub` , `iat` , `exp`) with short expiration periods (e.g., 2 hours).
- **Bcrypt Password Hashing:** Passwords are never stored in plain text. Credentials undergo cryptographic hashing using `bcrypt` with a workload salt factor of ***12*** rounds, providing resistance against dictionary and rainbow-table attacks.

12.2 Application Hardening & Input Sanitization

- **Express-Validator Data Middleware:** All incoming HTTP request bodies, headers, and URL parameters are sanitized and validated against explicit structural schemas to block malformed inputs.
- **Parameterized SQL Queries:** All database operations utilize parameterized queries and prepared statements via Object-Relational/Query Builder abstractions, preventing SQL Injection (SQLi) vectors.
- **Helmet HTTP Headers:** The `helmet` middleware secures HTTP headers by injecting protection defaults including Content Security Policy (CSP), X-Frame-Options (Clickjacking defense), HTTP Strict Transport Security (HSTS), and Strict-Transport-Security.
- **CORS Policy Enforcement:** Strict CORS controls restrict API execution exclusively to whitelisted frontend origins (e.g., CloudFront distribution domains).
- **Rate Limiting & Anti-Brute-Force:** IP-based rate limiting via `express-rate-limit` limits API endpoints to 100 requests per 15-minute window per IP, mitigating DDoS and brute-force attempts.

13. DevOps Implementation, Containerization & Orchestration

13.1 Docker Containerization Strategy

Both frontend and backend modules are packaged into standard Docker container images using multi-stage builds to minimize output image size and deployment surface area.

Backend Dockerfile Example

```
# Stage 1: Build Phase
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2: Production Execution Phase
FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --only=production
COPY --from=builder /app/dist ./dist
EXPOSE 5000
USER node
CMD ["node", "dist/server.js"]
```

13.2 Multi-Container Local Orchestration via Docker Compose

Local integration environments are orchestrated using Docker Compose to replicate production topologies locally.

```
version: '3.8'

services:
  backend-api:
    build:
      context: ./backend
      dockerfile: Dockerfile
    ports:
      - "5000:5000"
    environment:
      - NODE_ENV=development
      - PORT=5000
      - DB_HOST=mysql-db
      - DB_USER=root
      - DB_PASSWORD=secretpassword
      - DB_NAME=expense_tracker
      - JWT_SECRET=super-secret-key-12345
    depends_on:
      - mysql-db

  mysql-db:
    image: mysql:8.0
    ports:
      - "3306:3306"
    environment:
      - MYSQL_ROOT_PASSWORD=secretpassword
      - MYSQL_DATABASE=expense_tracker
    volumes:
      - mysql_data:/var/lib/mysql

volumes:
  mysql_data:
```

13.3 Kubernetes Production Orchestration Manifests

Production deployments are orchestrated on Kubernetes (EKS/K8s) clusters using declarative manifests for service routing, scaling, and secret management.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: expense-backend-deployment
  namespace: production
  labels:
    app: expense-backend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: expense-backend
  template:
    metadata:
      labels:
        app: expense-backend
    spec:
      containers:
      - name: backend-api
        image: 123456789012.dkr.ecr.us-east-1.amazonaws.com/expense-backend:v1.0.0
        ports:
        - containerPort: 5000
        envFrom:
        - configMapRef:
            name: backend-config
        - secretRef:
            name: backend-secrets
        resources:
          requests:
            memory: "256Mi"
            cpu: "250m"
          limits:
            memory: "512Mi"
            cpu: "500m"
---
apiVersion: v1
kind: Service
metadata:
  name: expense-backend-service
  namespace: production
spec:
  type: ClusterIP
  ports:
  - port: 80
    targetPort: 5000
  selector:
    app: expense-backend
```

13.4 CI/CD Pipeline Automation via GitHub Actions

Continuous Integration and Deployment workflows automate testing, artifact compilation, container image publishing, and deployment execution on code push events.

Mermaid Diagram Code: CI/CD Pipeline Workflow

```
graph LR Push[Code Push / Merge to Main] --> Checkout[1. Checkout Source Code] Checkout -->
LintTest[2. Run Linter & Unit Tests] LintTest --> BuildDocker[3. Build Production Docker Images]
BuildDocker --> PushECR[4. Push Image to Amazon ECR] PushECR --> DeployK8s[5. Rolling Update to
Kubernetes / EC2] DeployK8s --> SanityCheck[6. Automated Smoke & Health Check]
```

14. AWS Infrastructure Services Matrix

The cloud architecture leverages AWS services to achieve high availability, durability, performance, and monitoring.

AWS Cloud Service	Architectural Role & Functional Purpose
Amazon S3	Hosts compiled static React web application build assets and provides durable object storage for user transaction receipt images.
Amazon CloudFront	Global Content Delivery Network (CDN) providing low-latency distribution, HTTPS SSL termination, and static asset edge caching.
Amazon EC2	Provides scalable compute instances hosting Docker containers running the Node.js Express REST API layer.
Amazon RDS MySQL	Fully managed relational database running MySQL with Multi-AZ replication, automated backups, and encrypted storage.
Amazon CloudWatch	Collects runtime application logs, system performance metrics, and tracks key health statistics (CPU utilization, API latencies).
Amazon SNS	Sends push notifications, email alerts, and SMS messages when CloudWatch system alarms trigger.
AWS IAM	Enforces granular Role-Based Access Control (RBAC) and least-privilege security policies across all cloud services.

15. Implementation Details & Development Lifecycle

The project followed an agile methodology, divided into distinct engineering phases:

- **Phase 1: Frontend Development:** Architected the UI using React 19, TypeScript, and Vite. Implemented responsive layouts with Tailwind CSS, client-side routing via React Router v7, state management, and API integration using Axios.
- **Phase 2: Backend REST API Engineering:** Built an asynchronous Node.js Express API. Enforced secure authentication with JWT and bcrypt, implemented middleware pipelines for validation and security headers, and constructed database models.
- **Phase 3: Database Provisioning & Query Design:** Designed 3NF relational schemas in MySQL. Integrated connection pooling, indexed critical search paths, and configured schema migration scripts.
- **Phase 4: Containerization & Cloud Provisioning:** Created optimized multi-stage Dockerfiles and Docker Compose configurations. Provisioned AWS infrastructure including RDS MySQL instances, S3 buckets, CloudFront distributions, and IAM roles.
- **Phase 5: DevOps Pipeline Integration:** Authored GitHub Actions automation workflows to build container images, push artifacts to Amazon ECR, and execute automated deployments to production environments.

16. Quality Assurance & Software Testing Suite

The application underwent system, integration, and security testing to verify system reliability and requirements compliance.

Test Case ID	Feature / Subsystem Tested	Execution Procedure / Inputs	Expected Deterministic Result	Status
`TC-AUTH-01`	User Registration Validation	Submit invalid email format or weak password.	API returns HTTP 400 Bad Request with field validation errors.	**PASS**
`TC-AUTH-02`	User Login & JWT Issuance	Submit valid user email and password.	API returns HTTP 200 OK along with a valid signed JWT token.	**PASS**
`TC-EXP-01`	Create Expense Entry	POST expense payload with valid Bearer token.	Record inserted into RDS MySQL; returns HTTP 201 Created.	**PASS**
`TC-EXP-02`	Update Expense Record	PUT payload modifying transaction amount.	Database record updated; return HTTP 200 OK with updated state.	**PASS**
`TC-EXP-03`	Delete Expense Record	DELETE call targeting specific `expense_id`.	Record deleted from database; dashboard totals update immediately.	**PASS**
`TC-REP-01`	Category Analytics Generation	Trigger monthly aggregation query.	API returns aggregated spending breakdowns by category.	**PASS**
`TC-SEC-01`	Unauthenticated Endpoint Access	Access `/api/v1/expenses` without Authorization header.	Request blocked; API returns HTTP 401 Unauthorized.	**PASS**

17. Architectural Advantages & Key Differentiators

- **Universal Multi-Device Accessibility:** Cloud deployment and responsive client architecture ensure consistent access across web and mobile platforms.
- **Enterprise Security Architecture:** Multi-layer defense strategy—including JWT authentication, bcrypt hashing, rate limiting, and SQL injection prevention—ensures robust data protection.
- **High Availability & Infrastructure Elasticity:** AWS deployment with Multi-AZ RDS replication and Application Load Balancing guarantees system availability and automated resource scaling.
- **DevOps Automation:** Fully automated CI/CD pipelines streamline software testing, build delivery, and zero-downtime deployment execution.
- **Actionable Financial Analytics:** Converts raw transaction records into interactive visual charts and downloadable financial summaries.

18. Future Enhancements & Engineering Roadmap

- **Mobile Native Application:** Develop React Native mobile applications for iOS and Android, adding offline transaction caching and native push notifications.
- **Machine Learning & AI Expense Prediction:** Integrate AWS SageMaker ML models to auto-categorize transactions from receipt scans and forecast monthly spending trajectories.
- **Voice-Assisted Natural Language Interface:** Integrate Amazon Lex to enable voice-activated expense logging via natural language processing (NLP).
- **Automated Bank Feed Integration:** Integrate Open Banking APIs (e.g., Plaid) to automatically sync transactions from financial institutions.
- **Multi-Currency & Real-Time FX Conversions:** Support multi-currency transaction logging with dynamic foreign exchange rate conversion.

19. Conclusion & Engineering Summary

The **Cloud-Based Personal Expense Tracker using AWS & DevOps** project successfully demonstrates a cloud-native solution for modern personal financial management. By replacing fragmented, offline manual ledgers with a responsive React 19 web application, a Node.js Express REST API, and a managed Amazon RDS MySQL database, the platform delivers secure, highly available, and real-time expense tracking capabilities.

The implementation highlights key software engineering and DevOps principles: multi-layer security hardening (JWT, bcrypt, Helmet, input validation), containerized microservice execution (Docker, Kubernetes), automated build pipelines (GitHub Actions CI/CD), and resilient cloud infrastructure design (Amazon S3, CloudFront, EC2, RDS, CloudWatch, SNS). The platform establishes a robust baseline for cloud-native financial applications, achieving its core objectives of high availability, security compliance, operational automation, and seamless user experience.

20. References & Technical Documentation

1. **Amazon Web Services Documentation:** *AWS Cloud Architecture Guidelines, Amazon EC2, S3, RDS, and CloudFront Developer Guides.* Available at: `https://docs.aws.amazon.com/`
2. **React Documentation:** *React 19 - A JavaScript library for building user interfaces.* Available at: `https://react.dev/`
3. **Node.js & Express.js Architectural Guides:** *Building Production-Ready RESTful APIs with Node.js and Express framework.* Available at: `https://expressjs.com/`
4. **MySQL Technical Reference Manual:** *MySQL 8.0 Reference Manual, Query Optimization & Schema Design.* Available at: `https://dev.mysql.com/doc/`
5. **Docker Engine & Docker Compose Manuals:** *Containerization Principles & Multi-Container Application Orchestration.* Available at: `https://docs.docker.com/`
6. **Kubernetes Official Documentation:** *Container Orchestration Concepts, Deployments, Ingress Controllers, and Services Architecture.* Available at: `https://kubernetes.io/docs/`